Test unitaires avec PHPUnit

Table des matières

1. Introduction 2
2. Mise en place 2
3. Comment rédiger les test 6

1. Introduction

Un **test unitaire** consiste en une vérification sur une partie **isolée** du code, sortie de son contexte. On va par exemple vérifier qu'une méthode renvoie toujours quelque chose, qu'elle se comporte bien si je lui envoie de mauvais paramètres, et qu'elle ne casse rien le cas échéant.

On distingue trois types de tests, de manière générale :

Tests Unitaires
Teste une seule méthode, sans base de données, sans HTTP, sans dépendances réelles.

Tests d'Intégration
Teste plusieurs composants ensemble (service + repository, par ex.).

Tests Fonctionnels
Teste une route HTTP complète, du client jusqu'à la réponse.
À titre d'information, Symfony fournit également de quoi effectuer des tests fonctionnels grâce à WebTestCase, mais on n'en parlera pas aujourd'hui.

Pourquoi en écrire ? Un test unitaire vous évite de casser silencieusement du code existant à chaque modification. C'est une sécurité à la fois pour l'application mais aussi pour le développeur qui doit ajouter du code sur un projet existant parfois complexe : Vous ne casserez rien de l'existant même en rajoutant des features bien après.

2. Mise en place

Une commande composer suffit, et Symfony l'installe probablement par défaut si on lui demande de mettre en place les tests Unitaires quand le MakerBundle nous le demande.

```
composer require --dev symfony/test-pack
```

Cela nous crée le fichier phpunit.dist.xml dans lequel se trouvent un tas de paramètres auxquels on ne va pas toucher pour l'instant, à part pour rajouter ceci dans la balise **phpunit** :

```
        displayDetailsOnTestsThatTriggerNotices="true"
```

Cette précision dans la balise va nous permettre d'afficher même les erreurs bénignes comme les PHP Notice.

La vraie balise intéressante dans ce fichier est celle-ci puisqu'elle indique à PHPUnit où sont rangés les classes de test :

```
<testsuites>
    <testsuite name="Project Test Suite">
        <directory>tests</directory>
    </testsuite>
</testsuites>
```

Nous allons tester notre projet Doctofree par exemple.

Créez les fichiers nécessaire et rendez-vous dans le dossier **/tests/Unit/Entity/CabinetTest.php** pour y coller ceci :

```php
<?php
// tests/Unit/Entity/CabinetTest.php

namespace App\Tests\Unit\Entity;

use App\Entity\Cabinet;
use App\Entity\Medecin;
use PHPUnit\Framework\TestCase;

class CabinetTest extends TestCase
{
    private Cabinet $cabinet;

    protected function setUp(): void
    {
        $this->cabinet = new Cabinet();
    }

    public function testNomGetterAndSetter(): void
    {
        $this->cabinet->setNom('Cabinet du Parc');

        $this->assertSame('Cabinet du Parc', $this->cabinet->getNom());
    }

    public function testAdresseGetterAndSetter(): void
    {
        $this->cabinet->setAdresse('12 rue de la Paix, Paris');

        $this->assertSame('12 rue de la Paix, Paris', $this->cabinet->getAdresse());
    }

    public function testTelephoneGetterAndSetter(): void
    {
        $this->cabinet->setTelephone('0123456789');

        $this->assertSame('0123456789', $this->cabinet->getTelephone());
    }

    public function testMedecinsCollectionIsEmptyOnConstruct(): void
    {
        $this->assertCount(0, $this->cabinet->getMedecins());
    }

    public function testAddMedecinAddsToCollection(): void
    {
        $medecin = $this->createMock(Medecin::class);
        // On indique à PHPUnit que addCabinet() peut être appelé
        $medecin->method('addCabinet')->willReturnSelf();

        $this->cabinet->addMedecin($medecin);

        $this->assertCount(1, $this->cabinet->getMedecins());
```

```php
        $this->assertTrue($this->cabinet->getMedecins()->contains($medecin));
    }

    public function testAddMedecinDoesNotAddDuplicate(): void
    {
        $medecin = $this->createMock(Medecin::class);
        $medecin->method('addCabinets')->willReturnSelf();

        // On ajoute deux fois le même médecin
        $this->cabinet->addMedecin($medecin);
        $this->cabinet->addMedecin($medecin);

        // Il ne doit apparaître qu'une seule fois
        $this->assertCount(1, $this->cabinet->getMedecins());
    }

    public function testRemoveMedecinRemovesFromCollection(): void
    {
        $medecin = $this->createMock(Medecin::class);
        $medecin->method('addCabinet')->willReturnSelf();

        $this->cabinet->addMedecin($medecin);
        $this->cabinet->removeMedecin($medecin);

        $this->assertCount(0, $this->cabinet->getMedecins());
    }

    public function testToStringReturnsNom(): void
    {
        $this->cabinet->setNom('Cabinet Saint-Louis');

        $this->assertSame('Cabinet Saint-Louis', (string) $this->cabinet);
    }

    public function testIdIsNullByDefault(): void
    {
        // L'id est géré par Doctrine, il doit être null avant persistance
        $this->assertNull($this->cabinet->getId());
    }
}
```

Puis on va dire à PHPUnit de jouer tous les tests (--testdox nous permet d'afficher les messages de manière plus lisible)

```
php bin/phpunit --testdox
```

On obtient ceci :

```
PHPUnit 12.5.23 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.3.30
Configuration: /home/nicolas/wsldev/projects/doctofree/phpunit.dist.xml

.....E...                                                          9 / 9 (100%)

Time: 00:00.025, Memory: 16.00 MB

Cabinet (App\Tests\Unit\Entity\Cabinet)
 ✔ Nom getter and setter
 ✔ Adresse getter and setter
 ✔ Telephone getter and setter
 ✔ Medecins collection is empty on construct
 ✔ Add medecin adds to collection
 ✗ Add medecin does not add duplicate
   ┌─
   │ PHPUnit\Framework\MockObject\MethodCannotBeConfiguredException: Trying to configure method "addCabinets" which cann
ot be configured because it does not exist, has not been specified, is final, or is static
   │
   │ /home/nicolas/wsldev/projects/doctofree/tests/Unit/Entity/CabinetTest.php:60
   │ /home/nicolas/wsldev/projects/doctofree/vendor/phpunit/phpunit/phpunit:104
   └─
 ✔ Remove medecin removes from collection
 ✔ To string returns nom
 ✔ Id is null by default

ERRORS!
Tests: 9, Assertions: 9, Errors: 1.
```

On voit ici que mes tests se sont bien passées sauf un ☺

Regardons-les et corrigeons-les : une méthode mal écrite avec un S. Corrigez cette méthode en suivant l'indication de ligne de PHP Unit et rejouons la commande :

```
nicolas@MUSS:~/wsldev/projects/doctofree$ php bin/phpunit --testdox
PHPUnit 12.5.23 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.3.30
Configuration: /home/nicolas/wsldev/projects/doctofree/phpunit.dist.xml

.........                                                         9 / 9 (100%)

Time: 00:00.007, Memory: 16.00 MB

Cabinet (App\Tests\Unit\Entity\Cabinet)
 ✔ Nom getter and setter
 ✔ Adresse getter and setter
 ✔ Telephone getter and setter
 ✔ Medecins collection is empty on construct
 ✔ Add medecin adds to collection
 ✔ Add medecin does not add duplicate
 ✔ Remove medecin removes from collection
 ✔ To string returns nom
 ✔ Id is null by default

OK (9 tests, 10 assertions)
nicolas@MUSS:~/wsldev/projects/doctofree$
```

Tout de suite ça va mieux ☺ Mais ici nous n'avons débugué que notre propre test unitaire, pas un problème fondamental dans le code métier. Voyons voir comment ça se passe si on a une erreur dans notre code métier.

Allons modifier l'entité Cabinet.php : Imaginons qu'un développeur inexpérimenté décide de virer la vérification sur la fonction **addMedecin()** et la remplace par ce code :

```php
    // Cabinet.php
    public function addMedecin(Medecin $medecin): static
    {
        // Bug : on a supprimé le if (!$this->medecins->contains($medecin))
        // Le développeur a "simplifié" le code sans réfléchir
        $this->medecins->add($medecin);
        $medecin->addCabinet($this);
        return $this;
    }
```

Rejouons les tests et voyons ce qu'on obtient :

```
php bin/phpunit tests/Unit/Entity/CabinetTest.php --testdox
```

```
PHPUnit 12.5.23 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.3.30
Configuration: /home/nicolas/wsldev/projects/doctofree/phpunit.dist.xml

.....F...                                                         9 / 9 (100%)

Time: 00:00.008, Memory: 16.00 MB

Cabinet (App\Tests\Unit\Entity\Cabinet)
 ✔Nom getter and setter
 ✔Adresse getter and setter
 ✔Telephone getter and setter
 ✔Medecins collection is empty on construct
 ✔Add medecin adds to collection
 ✗ Add medecin does not add duplicate
   ┌─
   │ Failed asserting that actual size 2 matches expected size 1.
   │
   │ /home/nicolas/wsldev/projects/doctofree/tests/Unit/Entity/CabinetTest.php:67
   │ /home/nicolas/wsldev/projects/doctofree/vendor/phpunit/phpunit/phpunit:104
   └─
 ✔Remove medecin removes from collection
 ✔To string returns nom
 ✔Id is null by default

FAILURES!
Tests: 9, Assertions: 10, Failures: 1.
```

Ici le message nous dit qu'il s'attendait à trouver **un** médecin dans la collection (expected size 1) mais qu'il en a eu **2** en réalité (actual size 2)

C'est exactement le parcours classique d'un bug de production qui ne peut se détecter que grâce à des tests unitaires :

On travaille dans **src/**, puis on vérifie que tout fonctionne toujours bien en lançant les tests unitaires rédigés au préalable. Cela nous permettra de savoir immédiatement si on a cassé quelque chose ou non dans l'existant.

J'espère que vous comprenez donc bien leur rôle absolument crucial sur des projets complexes où plusieurs années de code et plusieurs dizaines de développeurs sont passées.

3. Comment rédiger les test

Chaque méthode déclarée dans cette classe de Test est une vérification des point clés que l'on veut vérifier sur notre classe Cabinet : Les getter et les setters, le fait que la collection Medecin soit vide au début, le fait qu'on ne puisse pas ajouter deux fois le même médecin…

Malheureusement, le MakerBundle ne va pas vraiment nous aider ici. Si on lui dit **oui** quand il nous demande si on veut générer les tests unitaires, il va simplement nous créer le fichier de test quasi-vide, comme ceci :

```php
TestControllerTest.php U ×
tests > Controller > TestControllerTest.php > ...
<?php

namespace App\Tests\Controller;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

final class TestControllerTest extends WebTestCase
{
    public function testIndex(): void
    {
        $client = static::createClient();
        $client->request('GET', '/test');

        self::assertResponseIsSuccessful();
    }
}
```

Les tests en questions seront à écrire à la main. Mais heureusement ce sont presque toujours les mêmes et ils obéissent à une logique de **nommage**.

Nommage des méthodes

Chaque classe de test étend TestCase et contient des méthodes préfixées test. Les noms de méthodes en **camelCase** seront ensuite interprétés dans le terminal pour être plus lisible, par exemple :

```php
public function testMedecinsCollectionIsEmptyOnConstruct(): void
```

Devient :

```
✔Medecins collection is empty on construct
```

Contenu des méthodes

Pour ce qui est du contenu des méthodes, on parle de pattern AAA pour la rédaction des tests. Voyez plutôt :

```php
namespace App\Tests\Unit;

use App\Service\MathHelper;
use PHPUnit\Framework\TestCase;

class MathHelperTest extends TestCase
{
    // Méthode setUp() : exécutée avant CHAQUE test
    protected function setUp(): void
    {
        $this->helper = new MathHelper();
    }

    // Nom = test + ce qu'on vérifie
    public function testAddReturnsCorrectSum(): void
    {
        // ARRANGE — préparer les données
        $a = 3;
        $b = 4;

        // ACT — appeler le code testé
        $result = $this->helper->add($a, $b);

        // ASSERT — vérifier le résultat
        $this->assertSame(7, $result);
    }
}
```

Le pattern AAA (Arrange, Act, Assert) est la structure universelle d'un test lisible. Respectez-la systématiquement.

Voici des exemples d'Assertions courrantes :

```php
// Égalité stricte (type + valeur)
$this->assertSame(42, $result);

// Égalité lâche (comme ==)
$this->assertEquals(42, $result);

// Booléens
$this->assertTrue($isValid);
$this->assertFalse($isEmpty);

// Null
$this->assertNull($value);

// Exception attendue
$this->expectException(\InvalidArgumentException::class);

// Tableau
$this->assertCount(3, $items);
$this->assertContains('foo', $items);
```

EXERCICE

- Générez moi un fichier de tests unitaires pour l'entité Spécialité et exécutez le lui et lui seul grâce à son chemin précisé dans la commande : php bin/phpunit tests/Unit/Entity/SpecialiteTest.php –testdox
- Faites pareil avec l'entité médecin